Atelier 2 : JavaScript dans le navigateur

GreenCart est une petite boutique en ligne de produits locaux. Dans l'atelier 1, vous avez écrit sa logique en Node : prix TTC, total du panier, frais de livraison. Aujourd'hui, cette logique arrive dans une vraie page web.

À la fin de l'atelier, la page affiche le catalogue, filtre les produits par catégorie, remplit un panier d'un clic et s'en souvient après un rechargement.

Rien de nouveau ici. Vous reprenez ce que le cours JavaScript vous a montré : le DOM, les événements, `localStorage`. Seul le bonus utilise `fetch`, et son squelette est fourni.

Ce que vous recevez

Fichier	Rôle	Vous le modifiez ?
`index.html`	La page, avec son style et ses zones vides prêtes à remplir	Non
`catalogue.js`	Les 12 produits GreenCart, dans la constante `CATALOGUE`	Non
`logique.js`	Les calculs, sans DOM. Le haut est fourni (atelier 1), le reste est à écrire	**Oui**
`app.js`	Ce qui touche à la page : affichage, clics, `localStorage`	**Oui**
`produits.json`	Le même catalogue en JSON, pour le bonus `fetch`	Non
`serveur-local.js`	Un petit serveur, pour le bonus `fetch`	Non
`verifier.js`	Le vérificateur de `logique.js`	Non

Les règles de GreenCart, déjà codées dans `logique.js` avec les noms de l'atelier 1 (`arrondirAuCentime`, `prixTTC`, `totalPanierTTC`, `fraisLivraison`, `montantAPayer`, `trouverParId`) :

- les prix du catalogue sont HT, la TVA est de 5,5 %
- le total TTC se calcule sur la somme des prix HT, sans arrondir ligne par ligne, puis s'arrondit au centime
- la livraison coûte 4,90 €, elle est offerte dès 40 € TTC, et un panier vide ne se livre pas
- on ne commande jamais plus que le stock
- le montant à payer, total TTC plus livraison, est arrondi au centime.

Un panier est un tableau de lignes. À l'atelier 1, vos lignes portaient le prix. Ici, elles renvoient au catalogue par son id. Chaque ligne donne l'id du produit et sa quantité :

```
[{ produitId: 1, quantite: 2 }, { produitId: 4, quantite: 3 }]
```

C'est pourquoi les fonctions du panier reçoivent aussi le catalogue : `totalPanierTTC(panier, catalogue)` et `montantAPayer(panier, catalogue)`.

Ouvrir la page

Double-cliquez sur `index.html`, ou glissez-le dans Chrome ou Firefox. Ouvrez aussi la console (F12, ou Ctrl+Maj+I, voir le cours JavaScript, slide 12). Gardez-la ouverte tout l'atelier : chaque erreur rouge vous dit où chercher.

Au départ, le catalogue est vide. C'est normal : c'est à vous de l'afficher.

Après chaque modification de `app.js` ou de `logique.js`, enregistrez et rechargez la page (F5).

Lancer le vérificateur

Dans un terminal, placez-vous dans ce dossier, puis :

```
node verifier.js
```

Le vérificateur teste les fonctions de `logique.js`. Il affiche une ligne par exercice : « OK », ou « À revoir » suivi de ce qui ne va pas. Relancez-le après chaque fonction écrite.

Il ne peut pas tester la page. Il affiche donc à la fin une liste de contrôle : ouvrez la page et cochez vous-même chaque point.

Comment travailler

Pour chaque exercice, deux temps :

1. **La fonction de `logique.js`**, jusqu'à ce que le vérificateur dise OK.
2. **La page, dans `app.js`**, qui appelle cette fonction. Vérifiez à l'écran.

Les exercices sont classés par priorité. Faites d'abord les trois **indispensables**. Passez ensuite au **souhaitable**, puis au **bonus** s'il vous reste du temps. Mieux vaut trois exercices qui marchent que cinq commencés.

Exercice 1 (indispensable) : afficher le catalogue

Ce qui est demandé

1. Dans `logique.js`, écrivez `libellePrix(produit)`. Elle renvoie le prix TTC suivi de l'unité, par exemple `"3,38 € / kg"`.
2. Dans `app.js`, complétez `afficherCatalogue(produits)`. Pour chaque produit, elle ajoute une carte dans la liste `<ul id="catalogue">` : le nom, le prix (avec `libellePrix`), la mention « Bio » si le produit est bio, et un bouton « Ajouter au panier ». Le modèle d'une carte est écrit en commentaire dans `index.html`.

Rappel

- Fonctions et `return` : cours JavaScript, slides 36 à 42.
- Boucle `for` sur un tableau : slides 34 et 49 à 52.
- `getElementById` et `querySelector` : slides 64 à 68.
- `createElement` et `appendChild` : slide 63.
- `innerHTML` : slide 70. Vous croiserez aussi `textContent` : il marche comme `innerHTML`, mais il écrit du texte simple, sans balises.

Résultat attendu

- `libellePrix` des Pommes Reinette (3,20 € HT au kg) renvoie `"3,38 € / kg"`.
- La page affiche 12 cartes. Celle des pommes ressemble à ceci :

```
Pommes Reinette
3,38 € / kg
Bio
[ Ajouter au panier ]
```

Indice

- `libellePrix` tient en une ligne. Les fonctions fournies font le travail : `prixTTC(produit.prix)` calcule le prix, `formaterEuros(...)` l'écrit avec une virgule et le symbole €. Il reste à coller `" / "` et `produit.unite` avec `+`.
- Dans `afficherCatalogue`, commencez par vider la liste : `listeCatalogue.innerHTML = ""`.
- Construisez le contenu de la carte dans une variable texte, morceau par morceau, puis donnez-le à `carte.innerHTML`.

À rendre : `logique.js` (fonction `libellePrix`) et `app.js` (fonction `afficherCatalogue`).

Exercice 2 (indispensable) : filtrer par catégorie

Ce qui est demandé

1. Dans `logique.js`, écrivez `filtrerParCategorie(catalogue, categorie)`. Elle renvoie un **nouveau** tableau qui ne contient que les produits de la catégorie demandée. Avec `"toutes"`, elle renvoie tous les produits. Le catalogue d'origine ne change pas.
2. Dans `app.js`, quand l'utilisateur choisit une catégorie dans la liste déroulante `<select id="choix-categorie">`, réaffichez le catalogue filtré.

Rappel

- Conditions `if` et comparaison `===` : slides 28 à 32.
- `push` sur un tableau : slide 49.
- La valeur d'un champ, `.value` : slide 72.
- `addEventListener` et l'événement `change` : slides 78 à 82.
- Un exercice proche du cours : l'exercice 32 « Filtrer les nombres pairs », slide 96.

Résultat attendu

- `filtrerParCategorie(CATALOGUE, "fruits")` renvoie les 2 fruits (Pommes Reinette, Poires Williams).
- Dans la page, choisissez « Légumes » : 4 cartes restent. Choisissez « Toutes » : les 12 reviennent.

Indice

- Partez d'un tableau vide, `const resultat = []`. Parcourez le catalogue avec `for`. Si la catégorie correspond, ou si on demande `"toutes"`, faites `resultat.push(...)`. Renvoyez `resultat`.
- Dans `app.js`, la fonction appelée par `change` tient en une ligne : `afficherCatalogue(filtrerParCategorie(catalogue, choixCategorie.value))`.

À rendre : `logique.js` (fonction `filtrerParCategorie`) et `app.js` (l'écoute de `change`).

Exercice 3 (indispensable) : ajouter au panier d'un clic

Ce qui est demandé

1. Dans `logique.js`, écrivez `ajouterAuPanier(panier, produit)`. Elle ajoute un exemplaire du produit et renvoie le panier mis à jour :
   - si le produit est déjà dans le panier, sa quantité augmente de 1
   - sinon, une nouvelle ligne `{ produitId: produit.id, quantite: 1 }` est ajoutée
   - la quantité ne dépasse jamais le stock. Un produit épuisé n'entre pas dans le panier.
2. Toujours dans `logique.js`, écrivez `nombreArticles(panier)`, qui additionne les quantités.
3. Dans `app.js` :
   - branchez le clic du bouton de chaque carte : il appelle `ajouter(produit)`
   - complétez `ajouter(produit)`
   - complétez `afficherPanier()` : une ligne par produit dans `<ul id="lignes-panier">`, et le nombre d'articles dans `<p id="compteur-panier">`.

Rappel

- L'événement `click` : slides 78, 81 et 84.
- Les recherches depuis un élément : slide 69. `carte.querySelector("button")` cherche dans la carte seulement.
- Exercices proches du cours : 17 et 18 (slide 93), 36 « Compteur » (slide 97).

Résultat attendu

- `ajouterAuPanier([], pommes)` renvoie `[{ produitId: 1, quantite: 1 }]`.
- `ajouterAuPanier([{ produitId: 1, quantite: 1 }], pommes)` renvoie `[{ produitId: 1, quantite: 2 }]`.
- `nombreArticles([{ produitId: 1, quantite: 2 }, { produitId: 4, quantite: 3 }])` renvoie `5`.
- Dans la page, deux clics sur les pommes affichent une seule ligne « Pommes Reinette x 2 », et « 2 articles » en haut.

Indice

- Dans `ajouterAuPanier`, cherchez d'abord le produit dans le panier avec une boucle `for`. Une variable `trouve`, à `false` au départ, vous dit ensuite s'il faut ajouter une ligne.
- Dans `afficherCatalogue`, écrivez `const produit = produits[i]` **dans** la boucle. Ainsi, chaque bouton garde son propre produit :

```
bouton.addEventListener("click", function () {
  ajouter(produit);
});
```

- Dans `afficherPanier`, le panier ne contient que des id. `trouverParId(catalogue, ligne.produitId)` vous rend le produit, donc son nom.

À rendre : `logique.js` (`ajouterAuPanier`, `nombreArticles`) et `app.js` (`afficherCatalogue`, `ajouter`, `afficherPanier`).

Exercice 4 (souhaitable) : un panier qui se souvient, des totaux, des produits épuisés

Ce qui est demandé

1. **Les totaux.** Dans `afficherPanier()`, écrivez le total TTC, la livraison et le montant à payer dans `#sous-total`, `#livraison` et `#total`. Les fonctions de calcul sont fournies : `totalPanierTTC`, `fraisLivraison`, `montantAPayer`, `formaterEuros`. Panier vide : laissez 0,00 € partout, il n'y a rien à payer.
2. **Les produits épuisés.** Dans `logique.js`, écrivez `estEpuise(produit)`, qui renvoie `true` quand le stock est à 0. Dans `afficherCatalogue`, grisez la carte d'un produit épuisé, désactivez son bouton et écrivez « Épuisé » dessus.
3. **Le panier dans `localStorage`.** Dans `logique.js`, écrivez `panierEnTexte(panier)` et `panierDepuisTexte(texte)`. Dans `app.js` :
   - à chaque ajout, enregistrez le panier
   - au chargement, relisez-le (remplacez `let panier = []`)
   - le bouton « Vider le panier » vide le panier, l'efface de `localStorage` et réaffiche.

Rappel

- La propriété `style` : slide 71.
- `JSON.stringify` et `JSON.parse` : slide 74.
- `localStorage`, `setItem`, `getItem`, `removeItem` : slides 75 et 76.
- Exercices proches du cours : 19 « Stockage local » (slide 93), 39 « Compteur persistant » (slide 98).

Résultat attendu

- Avec 2 kg de pommes : total TTC 6,75 €, livraison 4,90 €, à payer 11,65 €. Pas 6,76 € : la TVA s'applique à 6,40 € HT, ce qui donne 6,752 €, arrondi à 6,75 €. C'est la règle de l'atelier 1.
- Ajoutez 3 pots de miel d'acacia : total TTC 39,98 €, la livraison reste à 4,90 €. Un quatrième pot : elle passe à 0,00 €.
- Poires Williams et Yaourt au lait de chèvre sont grisés, leur bouton affiche « Épuisé » et ne réagit pas.
- Rechargez la page : le panier est toujours là.
- `panierDepuisTexte(null)` renvoie `[]`.

Indice

- `localStorage` ne range que du texte. D'où les deux fonctions : `panierEnTexte` avant `setItem`, `panierDepuisTexte` après `getItem`.
- La première fois, `localStorage.getItem(...)` renvoie `null`, car rien n'est encore enregistré. `panierDepuisTexte` doit alors renvoyer un panier vide.
- Pour griser : `carte.style.opacity = "0.6"`. Pour désactiver : `bouton.disabled = true`.

À rendre : `logique.js` (`estEpuise`, `panierEnTexte`, `panierDepuisTexte`) et `app.js`.

Exercice 5 (bonus) : charger le catalogue avec fetch

Ce qui est demandé

Aujourd'hui, le catalogue vient de `catalogue.js`, écrit en dur. Dans une vraie boutique, il vient d'un serveur. Chargez-le depuis `produits.json` avec `fetch`. Si le chargement échoue, affichez un message clair dans `#message` et gardez le catalogue de `catalogue.js`.

Le squelette est en bas de `app.js`, en commentaire. Décommentez-le et complétez les trois « À FAIRE ».

Attention : il faut un petit serveur local. Une page ouverte par double-clic a une adresse qui commence par `file://`. Le navigateur y interdit `fetch`, par sécurité. Le serveur est fourni :

```
node serveur-local.js
```

Lancez-le dans un terminal, dans ce dossier, puis ouvrez http://127.0.0.1:8642. Tapez bien cette adresse : `localhost` peut tomber sur un autre programme de votre ordinateur. Laissez ce terminal ouvert pendant que vous travaillez. Ctrl + C l'arrête. Il n'a besoin ni de `npm install` ni d'internet.

Si vous utilisez déjà l'extension **Live Server** de VS Code, elle marche aussi : clic droit sur `index.html`, puis « Open with Live Server ».

Rappel

`fetch`, `async` et `await` ne sont pas dans le cours. L'essentiel :

- `fetch("produits.json")` demande le fichier au serveur. La réponse prend un peu de temps.
- `await` attend cette réponse. Il ne s'écrit que dans une fonction marquée `async`.
- `await reponse.json()` transforme la réponse en tableau, comme `JSON.parse` (slide 74).
- `try` et `catch` attrapent l'erreur si le fichier est absent ou si le serveur ne répond pas.

Résultat attendu

- Servie par `node serveur-local.js`, la page affiche les 12 produits, sans message d'erreur.
- Changez le prix des carottes dans `produits.json` (2.0 au lieu de 1.8), rechargez : la carte affiche 2,11 € / kg.
- Renommez `produits.json`, rechargez : un message explique le problème, et le catalogue reste affiché.

Indice

`chargerCatalogue()` remplace les deux appels « Au chargement de la page ». Après le `try` et le `catch`, elle appelle elle-même `afficherCatalogue` et `afficherPanier`.

À rendre : `app.js`, avec `chargerCatalogue` décommentée et complétée.

Rendre votre travail

Rendez le dossier complet, avec `logique.js` et `app.js` modifiés. Avant de rendre :

- `node verifier.js` affiche OK pour les exercices faits
- la page s'ouvre sans erreur rouge dans la console
- vous avez coché la liste de contrôle du vérificateur.

Un exercice commencé mais pas terminé se rend aussi. Laissez un commentaire qui dit où vous vous êtes arrêté.